HDOC Day-64

Finding a Repeated Array Element Using Pointers and XOR

Problem Statement

Read the size and elements of a variable-length integer array nums[]. Pass its address to duplicate_element(). Find the single element repeated twice, while every other element occurs once, and display it using an efficient XOR approach.

Important assumption: XOR with 1, 2, ..., size-1 works only when an array of length size contains every integer from 1 through size-1, with exactly one duplicated value. The professor's three examples follow this pattern. For arbitrary distinct integers, XOR alone does not determine the duplicate without additional information.

Algorithm

1. Start.
2. Read size; reject a size less than 2 or greater than 10000.
3. Declare a variable-length array nums[size].
4. Read size integers; each must lie between 1 and size-1.
5. Call duplicate_element(nums, size), passing the array address.
6. Initialize result = 0. XOR each array element into result in one traversal.
7. XOR integers 1 through size-1 into result. All values that occur once cancel, leaving the duplicate.
8. Display the repeated element and stop.

Test Case Table

For each valid test case, enter the size first, then the listed elements. Each test case is a separate program run.

Test case	Size	Array elements / input	Expected output
1	5	1 3 4 2 2	2
2	5	3 1 3 4 2	3
3	6	1 4 5 2 3 4	4
4	2	1 1	1
5	5	1 2 3 4 4	4
6	5	1 1 2 3 4	1
7	6	5 4 3 2 1 5	5
8	7	2 6 3 1 4 5 6	6
9	1	(no elements)	Invalid size. Enter 2 to 10000.
10	5	1 2 3 4 5	Invalid element. Values must be from 1 to 4.

C Program

```
#include <stdio.h>

void duplicate_element(const int *nums, int size)
{
    int result = 0;
    int i;
    /* Assumes size n and values 1 through n-1, one repeated twice. */
    for (i = 0; i < size; i++)
        result ^= *(nums + i);
    for (i = 1; i < size; i++)
```

```
        result ^= i;
    printf("Repeated element: %d\n", result);
}

int main(void)
{
    int size, i;
    printf("Enter array size: ");
    if (scanf("%d", &size) != 1 || size < 2 || size > 10000) {
        printf("Invalid size. Enter 2 to 10000.\n");
        return 1;
    }
    int nums[size];
    printf("Enter %d elements (1 to %d, one repeated twice):\n", size, size - 1);
    for (i = 0; i < size; i++) {
        if (scanf("%d", nums + i) != 1 || nums[i] < 1 || nums[i] >= size) {
            printf("Invalid element. Values must be from 1 to %d.\n", size - 1);
            return 1;
        }
    }
    duplicate_element(nums, size);
    return 0;
}
```

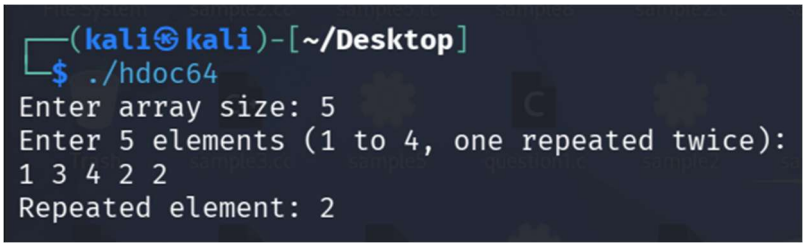
Compilation and Execution



Test Case 1

Input size: 5 Array: 1 3 4 2 2

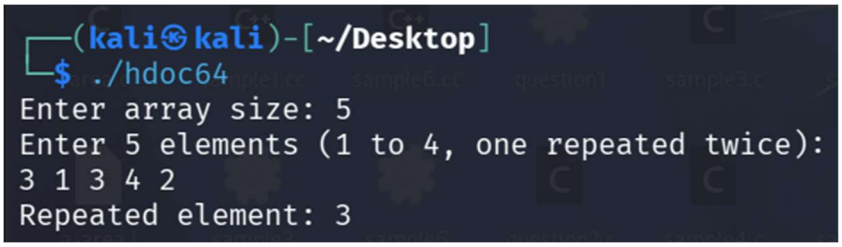
Expected output: Repeated element: 2



Test Case 2

Input size: 5 Array: 3 1 3 4 2

Expected output: Repeated element: 3



Test Case 3

Input size: 6 Array: 1 4 5 2 3 4

Expected output: Repeated element: 4



Test Case 4

Input size: 2 Array: 1 1

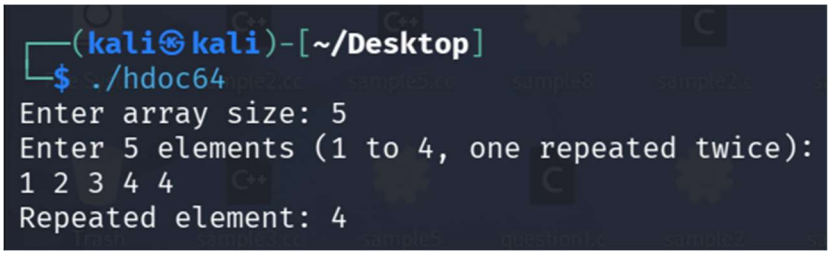
Expected output: Repeated element: 1



Test Case 5

Input size: 5 Array: 1 2 3 4 4

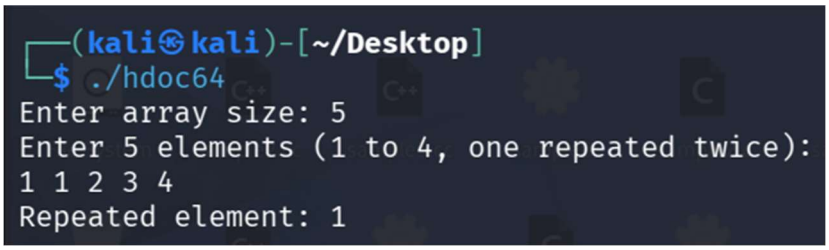
Expected output: Repeated element: 4



Test Case 6

Input size: 5 Array: 1 1 2 3 4

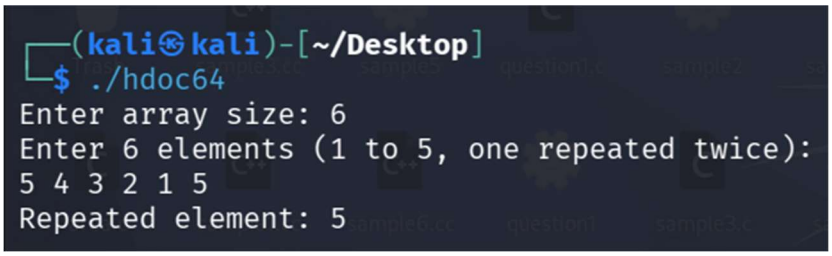
Expected output: Repeated element: 1



Test Case 7

Input size: 6 Array: 5 4 3 2 1 5

Expected output: Repeated element: 5



Test Case 8

Input size: 7 Array: 2 6 3 1 4 5 6

Expected output: Repeated element: 6

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc64
Enter array size: 7
Enter 7 elements (1 to 6, one repeated twice):
2 6 3 1 4 5 6
Repeated element: 6
```

Test Case 9

Input size: 1 Array: (no elements)

Expected output: Invalid size. Enter 2 to 10000.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc64
Enter array size: 1
Invalid size. Enter 2 to 10000.
```

Test Case 10

Input size: 5 Array: 1 2 3 4 5

Expected output: Invalid element. Values must be from 1 to 4.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc64
Enter array size: 5
Enter 5 elements (1 to 4, one repeated twice):
1 2 3 4 5
Invalid element. Values must be from 1 to 4.
```